

Winscale: An Image-Scaling Algorithm Using an Area Pixel Model

Chun-Ho Kim, Si-Mun Seong, Jin-Aeon Lee, and Lee-Sup Kim

Abstract—In this paper, we propose a new scaling algorithm, *winscale*, which performs the scale up/down transform using an area pixel model rather than a point pixel model. The proposed algorithm has low complexity: the algorithm uses a maximum of four pixels of an original image to calculate one pixel of a scaled image. Nevertheless, the algorithm has good characteristics such as *fine-edge* and *changeable smoothness*. We implemented a hardware design of *winscale* using an FPGA and displayed some test scenes in an liquid crystal display panel using a digital visual interface. The hardware cost and the image quality were compared with those of the conventional image scaling algorithms. It is proved that *winscale* has good scale property with low complexity. *Winscale* can be used in various digital display devices that need image scaling, especially in applications that require good image quality with low hardware cost.

Index Terms—Display-side scale, hardware implementation, image interpolator, image scale.

I. INTRODUCTION

DIGITIZING an image has many benefits in transmitting, processing, and storing the image data [1]. Recently digital display devices such as liquid crystal display (LCD) or plasma display panel (PDP) that directly use digital data in displaying images have become increasingly popular. However, bad transition exists still quite recently. That is, a flat panel display (FPD) interfaces to a column/row driver with digital signals and the frame buffer of a graphics card also stores digital pixel data. However, for transmission through analog cable, the digital image stream needs to be converted to analog signals using a D/A converter. Subsequently, for displaying to the FPD, the converted analog signals need to be recovered to digital signals using an A/D converter [2]. Through this transmission process, image quality may decrease and the image can be distorted by noise. To solve this, direct digital interfaces such as digital visual interface (DVI) that have high noise-immunity have been recently proposed and become popular in the display industry.

While the resolutions of image sources are various (e.g., VGA, SVGA, XGA, SXGA), the physical screen resolution of a digital display device is fixed. In the case where the resolution of a stream image generated by a host PC is different from the screen resolution of a digital display device, an image-scaling process is necessary. In the image-scaling process, image quality should be preserved as much as possible. It is also

desirable to implement a scaling step in dedicated hardware for averting performance degradation of CPU and/or graphics processing unit (GPU). Generally, the graphic card in a PC supports the scaling function. However, a display-side scaling is also possible. This has benefit of reducing computing overhead of a graphic card. In the applications where the scaling process must be performed at the display rather than the CPU or GPU, dedicated hardware implementation is necessary. In addition, it is desirable to reduce the cost of the hardware. Scaling streaming data without a frame buffer is a good method for lowering the cost. However, for vertical scaling, some line buffers are needed. Obviously, fewer line buffers are desirable in view of hardware costs.

The basic concept of image scaling is to resample a two-dimensional function on a new sampling grid [3], [4]. Already, many algorithms for image scaling are proposed. The simplest method is the *nearest neighbor* [5], which samples the nearest pixel from original image. It is regarded as the zero-order sample-and-hold and has good high frequency response, but degrades image quality due to aliasing. The most widely used method is the *bilinear* [6] which is regarded as the first-order sample-and-hold. In *bilinear*, the output pixel value changes linearly according to sampling position. There is more complex method called *bicubic* [7]. The weakness of *bilinear* and *bicubic* is blur effect causing bad high frequency response. Recently, many other methods using polynomial [8], adaptive [9], or correlative property [10] have been proposed. However, these methods have complex computation comparatively. In another method [11], the scale ratio is fixed at powers of two. It prohibits the method from being used in screen resolution change requiring a fractional scaling ratio.

In this paper, we introduce *winscale* algorithm, a new resampling method, which uses domain filtering utilizing area coverage of original pixels for calculating new pixels of a scaled image. This method has good high frequency characteristics and better image quality than *bilinear* method. This method is suitable for digital display devices in many aspects. For example, it preserves the edge characteristics of an image well, can handle streaming data directly, and requires only small amount of memory: four line buffers rather than a full frame buffer. We implemented the algorithm with an FPGA and displayed some test scenes on an LCD panel. The image quality and algorithm complexity are compared with those of conventional scaling algorithms.

II. WINSCALE: AN IMAGE-SCALING ALGORITHM

A. Basic Concept

Basically, the *winscale* algorithm uses an area pixel model and a domain filtering method for image scaling. That is, the algorithm uses an area-model rather than a point-model for a pixel evaluating. In our algorithm, the pixel shapes of the original and

Manuscript received May 7, 2002; revised December 1, 2002. This paper was recommended by Associate Editor L.-G. Chen.

C.-H. Kim and L.-S. Kim are with the Department of EECS, Korea Advanced Institute of Science and Technology, Daejeon, South Korea (e-mail: chkim@mvlsl.kaist.ac.kr; lskim@ee.kaist.ac.kr).

S.-M. Seong is with Curitel Communications Inc., Seoul, Korea (e-mail: gtr@curitel.com).

J.-A. Lee is with Mobile Design P/J, SoC R&D Center, DSN, Samsung Electronics, Suwon, Korea (e-mail: jalee@samsung.com).

Digital Object Identifier 10.1109/TCSVT.2003.813431

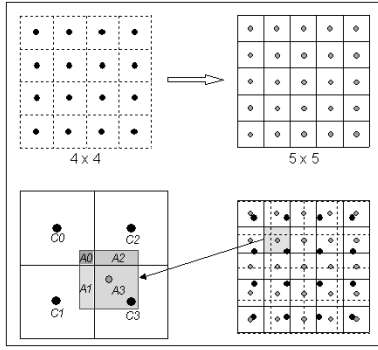


Fig. 1. General concept of scale up.

the scaled image are treated as rectangles and pixel intensity is evenly distributed in rectangle area. Therefore, each new pixel of the scaled image is obtained by weighted-averaging the pixel values of the original pixels with area coverage ratio. For explanation, we first define some terminologies. A *source image* is the original image before scaling up/down. *Target image* refers to the scaled image. The region of the target pixel being calculated currently is called a *filter window*. In our algorithm, scale transform can be done along the scan-line direction for streaming data.

Fig. 1 shows an example of the image scale up process (from a resolution of 4×4 pixels to a resolution of 5×5 pixels) to illustrate the basic concept. In the scale up process, the number of source pixels overlapped by a target pixel is one, two, or a maximum of four because the area of a target pixel is less than that of a source pixel as shown in the lower-left of Fig. 1. The pixel values of four source pixels overlapped by a target pixel are $C0-C3$. Overlapped regions of the four source pixels by the target pixel are $A0-A3$. We assume that the area of a source pixel is 1.0. Therefore, the pixel value P of a target pixel is obtained by

$$\begin{aligned} P &= SF \times (A0 \cdot C0 + A1 \cdot C1 + A2 \cdot C2 + A3 \cdot C3) \\ &= w0 \cdot C0 + w1 \cdot C1 + w2 \cdot C2 + w3 \cdot C3 \\ SF &= 1/\text{area of filter window.} \end{aligned} \quad (1)$$

Then, the filter window is moved for the next target pixel in the row or column direction. If the filter window is perfectly included in one source pixel region, sampling has the *nearest neighbor* property: the pixel value becomes equal to that of source pixel. This property makes good high frequency response and reduces blurring effect. If the filter window is spread over multiple source pixels, sampling has the linear property like *bilinear*: smoothness is added. Therefore, *winscale* inherits two good properties of both nearest and linear: good high frequency response and good smoothness. The scale factor (SF) compensates the reduced intensity due to the small area (less than 1.0) of a target pixel. The value of SF will be larger than 1.0 for the scale-up case.

Fig. 2 provides an example of the image scale-down process (from a resolution of 5×5 pixels to a resolution of 4×4 pixels). The basic principle of the scale-down process is almost the same as the scale-up process, except for the employment of prescaling and the size of a filter window. If the scale-down ratio is larger than 2.0, the *prescaler* performs the scale-down by evenly weighted summation with two's power ratio to adjust the scale down ratio

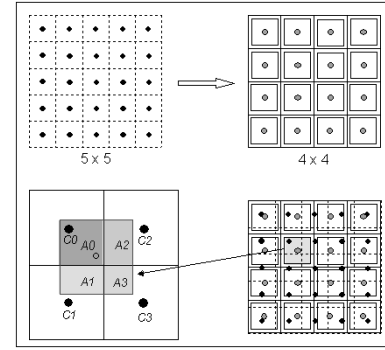


Fig. 2. General concept of scale down.

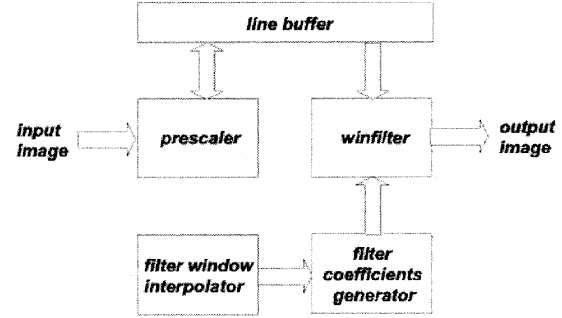


Fig. 3. Functional block diagram of winscale.

to be between 1 and 1/2 before the main scale-down process. By this prescale process, it is ensured that all original pixels are used at least one time. That is, all original pixels affect the scaled image irrespective of the scale-down ratio. The filter window's width and height are shrunk to 1.0 to guarantee overlapping with the maximum of four source pixels. Therefore, in the case of scale down, the filter window's incremental value for the next target pixel will be larger than 1.0 and the value of SF will be 1.0 without respect to the scale ratio.

B. Detailed Functions

Fig. 3 shows a functional block diagram of *winscale*. It is composed of five blocks: the *prescaler* block (used only for the scale down ratio larger than 2.0), the *line buffer* block (used for vertical streaming buffer), the *winfilter* block (executes filtering of weighted summation), the *filter window interpolator* block (increases or decreases filter window coefficients), and the *filter coefficient's generator* block (calculates weighting factors). The *prescaler* is composed of shifters and adders for scaling of two's power along the horizontal and vertical directions. The *winfilter* is composed of four multipliers and three adders for weighted summation of four pixels.

SF is calculated from the scale ratio just once at the setup time. Filter window coefficients are six variables. These are the center coordinates (winX , winY) and boundary coordinates (winLX , winRX , winTY , winBY) of the filter window. These six filter window coefficients are initially set to constant values for the first target pixel according to scale ratio and incremented for the next target pixel by six additions in the filter window interpolator block. The incremental values for horizontal and vertical direction are winW and winH respectively. The winW and

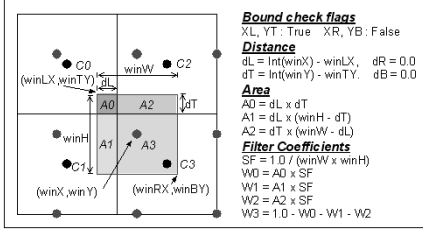


Fig. 4. Calculation of areas coverage and filter coefficients.

winH are obtained by

$$\begin{aligned} \text{winW}(\text{filter window width}) &= 1/\text{horizontal scaleup ratio} \\ \text{winH}(\text{filter window height}) &= 1/\text{vertical scaleup ratio}. \end{aligned} \quad (2)$$

The weighting factors ($W0-W3$) are calculated by multiplying area coverage ($A0-A3$) of the filter window and SF . $A0-A3$ are calculated from dL , dR , dT , and dB , which are a fractional part of the boundary coordinates of the filter window. It is shown in Fig. 4.

Scaling is processed line by line: vertical movement occurs after one horizontal completion. If the direction of the filter window movement is horizontal, the next coordinates of the filter window are obtained from the addition of the previous filter window's x coordinate and the filter window's width. Also, if the direction of filter window movement is vertical, the next coordinates of the filter window are obtained from the addition of the previous filter window's y coordinate and the filter window's height.

By comparing these four boundary coordinates and the center position of the filter window, we can check the boundary coverage of the filter window on the input pixel field. According to this result, the filter window interpolator generates four one-bit filter coverage flags (XL, XR, YT, YB). These flags are used to denote whether a covered pixel exists in each four directions (left, right, top, bottom) of the center (winX, winY) pixel or not. That is, if the XL flag is set, it means that the left pixel is covered by the filter window. In Fig. 4, the left and the top pixels are covered by the filter window: XL and YT flags hold the TRUE (1) value and XR and YB flags hold the FALSE (0) value. By referring to these flags (XL, XR, YT, YB), parameters dL and dT are calculated easily to obtain area coverage ($A0-A3$) of the filter window. The following step is multiplication of SF with the area coverage ($A0-A3$). The outcomes are weighting factors ($W0-W3$). Using these weighting factors and original pixels ($C0-C3$), the filtered output pixel (P) is finally calculated.

C. Overlap Stamping

In the case where the scale-up ratio is an integer, the *winscale* algorithm has a drawback. That is, the region of the filter window is overlapped with just single pixel of the source image. Therefore, a weighting factor is 0 or 1 and the *winscale* algorithm becomes same as the case of *nearest neighbor* using a point pixel model. In this case, the aliasing in edges will increase. To overcome this drawback, we introduce an overlap-stamping method, which expands the filter window to be overlapped with multiple source pixels as shown in Fig. 5 for in-

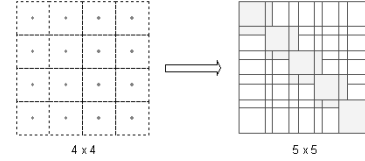
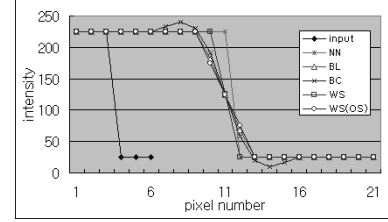
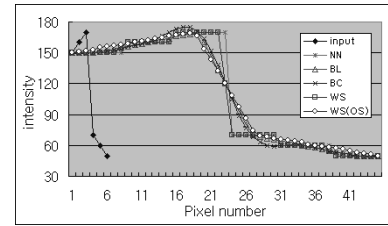


Fig. 5. Filter window overlapping in overlap stamping.



(a)



(b)

Fig. 6. Steppedge response by scale up. (a) Abruptly from 225 to 25. (b) Gradually, from 150 to 170, drop abruptly to 70, and then change gradually to 50.

creasing smoothness. In the case of scale up from $N \times N$ to $M \times M$ pixels ($M > N$), the area of the filter window is expanded to 1.0 and the incremental value of the filter window is set to $(N - 1)/(M - 1)$. By overlap stamping, the smoothness of the scaled image increases, and the point-sampling problem of integer scaling ratio is overcome.

D. Fine-Edge Characteristic and Changeable Smoothness

Human eyes are sensitive to the edges of an image. Also, it is dependent on the kinds of images. That is, smooth edges look good in natural images such as landscape scenes, animal pictures, and so on, but fine edges, rather than smooth edges, look better in some cases such as text images. Therefore, it is advantageous to control the smoothness of an image with consideration about the kind of image in image processing like scaling. In the *winscale* algorithm, fine edges are well preserved, as shown in Fig. 6. In Fig. 6(a) and (b), the steppedge is moved to the right by scale up. The well-preserved edges are helpful for a text image, but may not be for a natural image requiring high smoothness. Smoothness of edges can be increased by overlap stamping for the natural image. Therefore, we can control the smoothness of a target image by applying overlap stamping or not. To apply overlap stamping, it is necessary only to change the size of the filter window and incremental value. This does not require hardware architecture modification. Filter window coefficients and incremental values for horizontal and vertical directions are initially calculated and stored in memory. Thus, to apply overlap stamping is merely to change the coefficients value at setup time. With this, we can change the smoothness of a scene without any hardware modifications.

TABLE I
COMPARISON OF OPERATIONS PER PIXEL

	<i>Nearest Neighbor</i>	<i>Bilinear</i>	<i>Winscale</i>	<i>Bicubic</i>
Pixel moving	2 add	2 add	6 add	2 add
Weighting factor calculation	0 mult 0 add	3 mult 2 add	6 mult 2 add	16 mult 36 add
Pure Filter operations	0 mult 0 add	4 mult 3 add	4 mult 3 add	16 mult 15 add
Memory access	1 read 1 write	4 read 1 write	4 read 1 write	16 read 1 write

III. FPGA TEST

Winscale was implemented in hardware using an FPGA, and some test scenes were displayed in an LCD panel using DVI. In practical hardware implementation, we limit the precision of filter coefficients as 30 bits, which is sufficient for filter coefficients calculation. We need a maximum of four input pixels for the filtering process to obtain one pixel of target image. Only two lines of input image are needed for calculation of a target pixel. Therefore, image scaling can be done with only four line buffers (two lines for currently used pixels and two lines for buffering the next vertical pixels) regardless of the scale up/down ratio.

The *winscale* algorithm was described in a hardware description language (HDL) and tested using an FPGA. The NAND-equivalent gate count for *winscale* was about 29 000 except for line buffer memory. Resolution of the LCD was XGA (1024×768) and resolution of the source image was VGA (640×480). The required operation frequency was 65 MHz. The source image transmitted by DVI was scaled up by the *winscale* algorithm mapped in FPGA and displayed on the LCD.

IV. SCENE COMPARISONS

We compared *winscale* algorithm to various conventional algorithms in terms of image quality and algorithm complexity. There are many image scaling algorithms, ranging from expensive algorithms like the adaptive method to inexpensive algorithms like the simple repetition method. We used the *nearest neighbor*, *bilinear*, and *bicubic* algorithms for comparison. These algorithms are all implemented by C language. Precision limits were adopted in these algorithms for fair comparisons like in the hardware implementation of *winscale*. Other algorithms considering minimum loss or adaptive filtering [12], [13] were excluded in the comparison because they are unsuitable in terms of hardware comparison (they require much larger hardware resources).

The simple repetition algorithm has inexpensive hardware costs. However, we cannot obtain good image quality from this algorithm because repetition results in blocky effects. Complex algorithms are expensive in hardware costs. The *bilinear* algorithm is similar to the *winscale* algorithm in hardware structure and costs. Operations of comparative algorithms for filtering operation per one pixel are shown in Table I.

Quantitative comparison can be done by the root mean square error (RMSE) of the luminance (Y) component. For RMSE comparisons, two methods are used. One is scaling down from XGA (1024×768) to VGA (640×480) and then scaling up

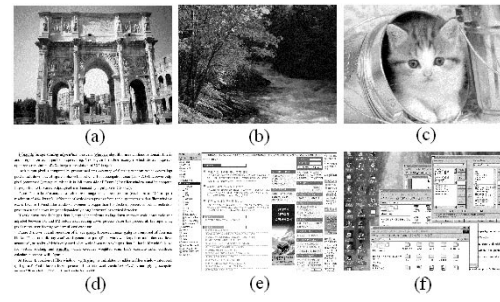


Fig. 7. Six test images for RMSE comparisons.

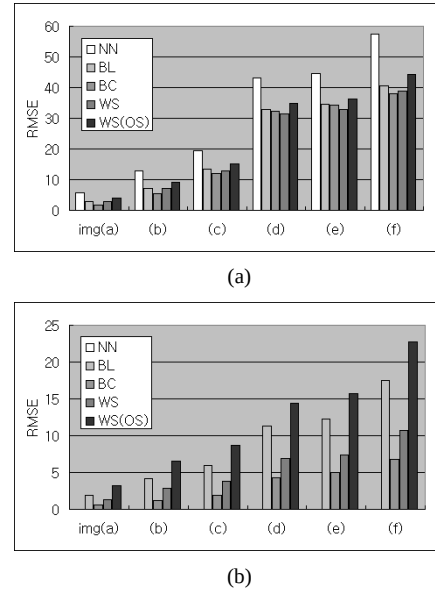


Fig. 8. RMSE comparisons. RMSE of luminance(Y) channel for (a) up-after-down images and (b) down-after-up images.

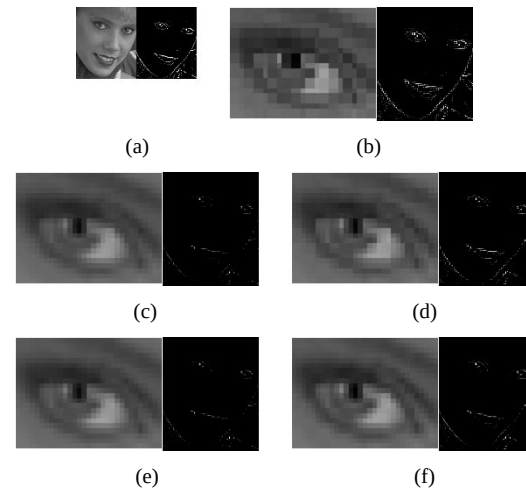


Fig. 9. Natural image scale-up and their edgmap. (a) Original. (b) *Nearest neighbor*. (c) *Bilinear*. (d) *Winscale*. (e) *Winscale* (overlap stamping). (f) *Bicubic*.

back to XGA. The other is scaling up from VGA to XGA and then scaling down to VGA. We used six images (three natural and three text-like images) shown in Fig. 7 in this step. Fig. 8 shows RMSE comparisons. In Fig. 8(b), the RMSE values of the scaled images by *nearest neighbor* have zero because scale down-after-up by point sampling returns back to the original.

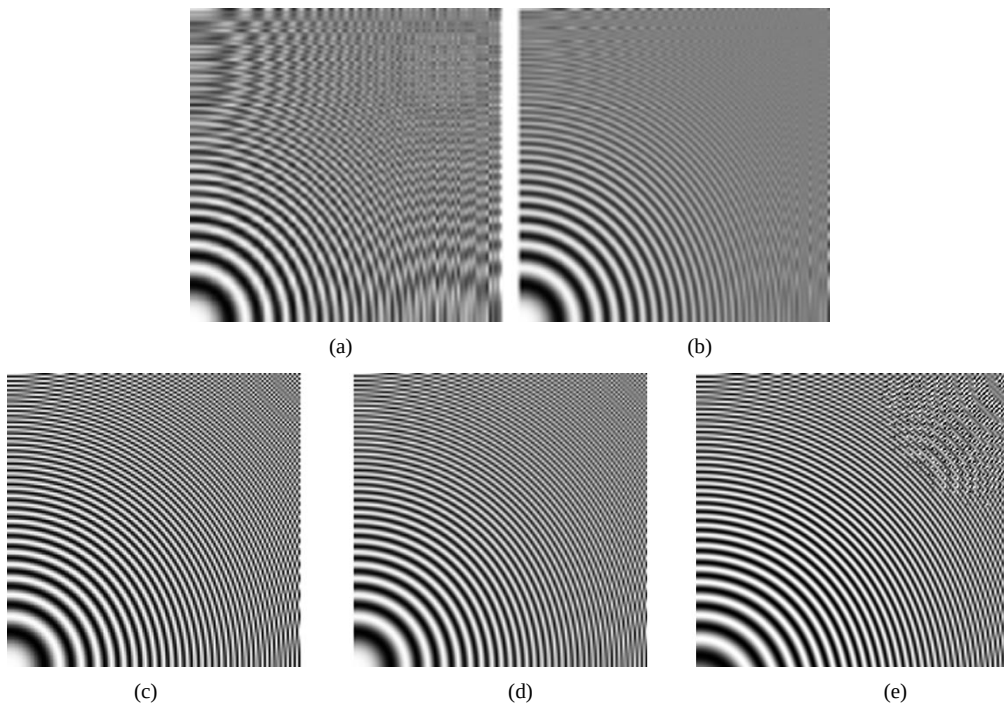


Fig. 10. Zoneplate image scale up. (a) *Nearest neighbor*. (b) *Bilinear*. (c) *Winscale*. (d) *Winscale (overlap stamping)*. (e) *Bicubic*.

From these results, we can see that *winscale* algorithm achieves the same as or better image quality than the *bilinear* algorithm with comparative complexity. As shown in Fig. 8, RMSE values of *winscale* are lower than those of *bilinear* algorithm. However, it is not sufficient to compare only RMSE. Qualitative comparison is also necessary though it is subjective and dependent on human vision. The *winscale* algorithm has good edge characteristics. Because domain filtering uses a small number of pixels (1, 2, or 4), blurring can be reduced. To see fine-edge characteristics, edge maps of original and scaled images are shown in Fig. 9. In Fig. 9, images are partially captured and the right eye of the woman is zoomed in image scaled up from a VGA to an XGA image. Obviously, *nearest neighbor* and *winscale* preserve edges well. This fine-edge characteristic is due to good high-frequency response. Also, a zoneplate image is used to compare frequency response of the various scaling methods. As shown in Fig. 10, we find that *winscale* has frequency characteristics as good as those of *bilinear*. The bad high-frequency response of *bicubic* is due to high smoothness caused by filtering using multiple source pixels (16 pixels).

V. CONCLUSION

In this paper, we proposed a new image-scaling algorithm, *winscale*, and verified the algorithm with a hardware implementation for display-side scaling using FPGA. *Winscale* is composed of five functional blocks (prescaler, line buffer, winfilter, filter window interpolator, and filter coefficients generator) and has about 29 000 NAND-equivalent gate counts after synthesis. The scale algorithm is mapped in an FPGA and some test scenes are displayed in an LCD panel using DVI. The proposed *winscale* algorithm has better image quality (lower RMSE) than the *bilinear* algorithm, which has comparable complexity. In addition,

the *winscale* algorithm requires only four line buffers for vertical scaling in the stream processing. As a result, *winscale* is an effective image-scaling algorithm for applications that require good image quality and low computational complexity.

REFERENCES

- [1] A. K. Jain, *Fundamentals of Digital Image Processing*. Englewood Cliffs, NJ: Prentice-Hall, 1989.
- [2] L.-M. Joshua, "Flat panel display interface technologies," in *Silicon Graphics*. Mountain View, CA: Silicon Graphics, Inc., 1998.
- [3] W. K. Pratt, *Digital Image Processing*. New York: Wiley-Interscience, 1991.
- [4] J. A. Parker, R. V. Kenyon, and D. E. Troxel, "Comparison of interpolation methods for image resampling," *IEEE Trans. Med. Imag.*, vol. 2, pp. 31–39, 1983.
- [5] S. Fifman, "Digital rectification of ERTS multispectral imagery," in *Proc. Significant Results Obtained From Earth Resources Technology Satellite-1*, vol. 1, 1973, pp. 1131–1142.
- [6] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*. Reading, MA: Addison-Wesley, 1992.
- [7] H. S. Hou and H. C. Andrews, "Cubic splines for image interpolation and digital filtering," *IEEE Trans. Acoust., Speech, Signal Processing*, vol. ASSP-26, pp. 508–517, 1978.
- [8] S. Andrews and F. Harris, "Polynomial approximations of interpolants," in *Conf. Record 33rd Asilomar Conf. Signals, Systems, and Computers*, vol. 1, 1999, pp. 447–451.
- [9] N. Shezaf, H. Abramov-Segal, I. Sutskover, and R. Bar-Sella, "Adaptive low complexity algorithm for image zooming at fractional scaling ratio," in *21st IEEE Conv. Electrical and Electronic Engineers*, Tel Aviv, Israel, 2000, pp. 253–256.
- [10] H. C. Kim, B. H. Kwon, and M. R. Choi, "An image interpolator with image improvement for LCD controller," *IEEE Trans. Consumer Electron.*, vol. 47, pp. 263–271, May 2001.
- [11] S. Carrato and L. Tenze, "A high quality $2 \times$ image interpolator," *IEEE Signal Processing Lett.*, vol. 7, pp. 132–134, June 2000.
- [12] M. Unser, "Enlargement or reduction of digital images with minimum loss of information," *IEEE Trans. Image Processing*, vol. 4, pp. 247–258, Mar. 1995.
- [13] A. M. Darwish, M. S. Bedair, and S. I. Shaheen, "Adaptive resampling algorithm for image zooming," in *Proc. Inst. Elect. Eng. Visual Image Signal Process*, vol. 144, Aug. 1997, pp. 207–212.